

Inigo Quilez :: articles :: multiresolution ambient occlusion - 2012



Intro

While we wait the technology to progress enough for realtime global illumination to be possible, we have to find hacks if we want to create believable images. Over the years, a countless amount of hacks have been developed, from high quality precomputed light probes encoded in spherical harmonics and stored in volumetric data structures (like octrees or tetrahedra grids) or plain textures, to dynamic approximations with raymarching in low resolution volumes, or mini-rasterization of pointclouds, to screen space hacks like screen space ambient occlusion (SSAO) and color bleeding (SSCB).

From all, the most popular technique among the hobbyists has been the SSAO, for it doesn't need the tedious development of baking tools at all, but a simple pixel shaders with a handful of lines of code. In my opinion, it's also the most abused and probably wrongly applied effect (closely followed by bloom), but more of that later.



Realtime (multiresolution) ambient occlusion (OpenGL)



Final realtime image (OpenGL)

Besides halos and performance, the main issue that limits the usability of SSAO is the range of distances at which the technique produces results that can be interpreted as occlusion. Indeed, high neighbor sampling kernels not only are expensive but actually do not represent occlusion very well, due to the lack of second and third depth layers to sample from. At the same time if the kernel is made too small the illusion of occlusion disappears as well, and all that remains is an ugly edge enhancer which gives it all a horrible cartoon shading effect. So, it seems that traditional SSAO is optimal only to solve medium frequency occlusion, while big/small size occlusion (low/high frequencies) need something else.

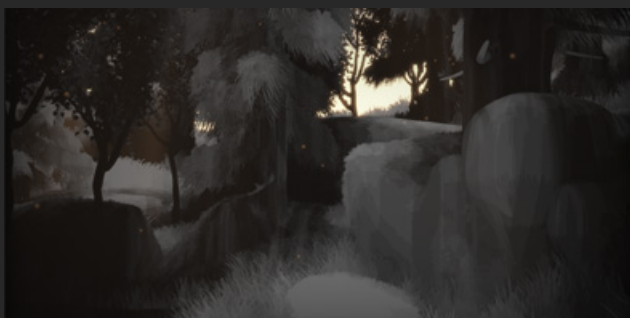
In this article we'll see one approach to solve the three types of occlusion for one specific type of scene, although the ideas can be (and probably has been) in other contexts.



High frequency occlusion



Medium frequency occlusion



Low frequency occlusion

Medium frequency ambient occlusion

As said, this can be solved with the traditional SSAO technique, which is more than documented enough, in all its flavours, variations and versions. I'd say, almost every developer has its own version of SSAO. In my case, I basically write a new one from scratch for every demo or shot in a demo. They are all very similar, though, probably something similar to this:

```
uniform vec3      unKernel[16];
uniform sampler2D  unTexZ;
uniform sampler2D  unTexN;
uniform sampler2D  unTexR;

float ssao( in vec2 pixel )
{
    vec2 uv = pixel*0.5 + 0.5;
    float z  = texture2D( unTexZ, uv ).x;           // read eye linear z
    vec3 nor = texture2D( unTexN, uv ).xyz;         // read normal
    vec3 ref = texture2D( unTexD, uv ).xyz;         // read dithering vector
```

```

// accumulate occlusion
float b1 = 0.0;
for( int i=0; i<16; i++ )
{
    vec3 of = orientate( reflect( unKernel[i], ref ), nor );
    float sz = texture2D( unTexZ, uv+0.03*of.xy).x;
    float zd = (sz-z)*0.2;
    b1 += clamp(zd*10.0,0.1,1.0)*(1.0-clamp((zd-1.0)/5.0,0.0,1.0));
}

return 1.0 - 1.0*b1/16.0;
}

```

In this case I'm using the oldest SSAO known (with the usual reflection based dithering) plus sample orientation based on the normal (which makes better use of the samples, plus brings normal mapped detail to the occlusion signal). The *orientate()* function flips the input vector if its dot product with the normal is negative. I'm putting the gbuffer normals and eye linear z in two different textures for less bandwidth usage during the z sampling loop.

But of course here you can and should apply your preferred SSAO variation, with your dithering method of choice (none, per pixel 2d rotation matrix or random reflection vector), your preferred halo removal trick, you can do this at a quarter resolution and then bilaterally upsample, you can use the normal in a more fancy way than i did, you can do smooth volumetric occlusion, etc etc etc. You pick your flavour, more than 5 years of documented SSAO experiments should be enough for anybody :D



Medium frequency occlusion - standard SSAO

High frequency ambient occlusion

High frequency occlusion, in this scene I'm dealing with and in many others, can be regarded as the occlusion an object casts on itself. That means that the occlusion signal can probably come from the model itself, either in form of backed information or with a procedural description.

In case of nature, a procedural description for occlusion makes a lot of sense. The same procedural methods that grow the grass, bushes, tree trunks and branches or canopies can take care of producing an occlusion signal too that matches the procedurally generated geometry. In other words, since the code/procedure generates the geometry, it knows about it (like, where it is, how big it is), therefore it can produce plausible occlusion information.



High frequency occlusion - procedural occlusion

For example, when generating leaves for a tree canopy, those in the inner part of the canopy can get darker than those outside (this is a very simplistic approach, but it shows the concept). The code that procedurally grows the grass can shade blades darker or brighter depending their relative position to the clump they belong to. The trunk of a procedural tree can get darker in the areas where the amount of branches is higher, etc etc.

These might seem like naive observations and way to do high frequency occlusion, but in practice it works very well.

If for some esoteric reason you were looking for something more "physically realistic", you could bake the local high frequency occlusion of your models per vertex or in a lightmap.

The idea of course is to not bake all the occlusion we can procedurally. Sometimes it's tempting to be too clever and generate procedurally or bake as much occlusion as possible, but we don't want to do that, for the point is to make occlusion realtime and as automatic as possible as part of the lighting solution. So, we want

to generate/bake only the occlusion that the medium frequency realtime ambient occlusion (SSAO in our case) cannot capture.

Now we have to combine the new high frequency occlusion with the classic medium frequency occlusion. The very naive multiplication of signals works just beautifully:



Medium frequency occlusion (SSAO) multiplied with the high frequency occlusion (procedural)

Low frequency ambient occlusion

This is the occlusion which comes from objects too far for the SSAO to capture. One could bake the information in low resolution lightmaps, but instead, game/demo specific tricks can be used which are cheaper, don't require tools or an export pipeline, and can be realtime.

In the case of the scene that we are currently playing with, we have an outdoors scene, so what we basically want to see is the trees and big rock formation casting occlusion around and beneath them. One way to get something like that easily is by rendering a shadowmap with a vertical direction that captures all of the trees and rocks, then do a super blurry/lp-filtered lookup. That sort of makes the job of darkening the areas below the trees and leak a bit of darkness to the neighboring objects.



Low frequency occlusion - blurry shadowmap

Probably three more blurry shadowmaps could be additionally rendered rotated 120 degrees azimuth and 45 degree in altitude and get a much better approximation to low frequency ambient occlusion. But remember that perfect ambient occlusion doesn't make a good image anyway, it's still a hack in itself far from proper light propagation.

Despite I can imagine something more elaborate that a simple multiplication for combining the low occlusion with the medium and high frequency occlusion, seems like the multiplication isn't that bad really:



High (procedural), medium (SSAO) and low (shadowmap) frequency occlusion multiplied

Results

Now that we have a beautiful occlusion signal, it's time to use it in our lighting pass.

One mistake most beginners do is to use the occlusion as a multiplier to the whole lighting equation. That's is **NOT A GOOD IDEA**, and among other issues, it produces horrible ghosting effect that have become sadly regular in most realtime rendering efforts. Now, even if you had perfect halo free ambient occlusion, multiplying the whole lighting equation with it would still be **A REALLY BAD IDEA**.

Ambient occlusion is, after all, an occlusion factor for the ambient lighting. "Ambient" is a bit of a vague term, but in the context of classic OpenGL-kind of rendering, it probably means all lights but the key lights, meaning the sky dome lighting, all of the bounce lighting, and the rims. In practice, it means that you probably only want your occlusion signal to affect all the lights witch are not strong direct lighting sources. In our case, the occlusion will affect everything but the sun light. It's all up to you to choose how much occlusion you leak in each light, and there are many ways to do this. You can even colorize the occlusion signal with a color ramp to bring some extra richness to the image.



Multiresolution ambient occlusion used with ambient lighting

If you allow me, here goes another advice: do not implement the infamous "hemispherical occlusion", the trick of modulating occlusion with the vertical component of the surface's normal. That's probably a really bad idea because, again, we want to use occlusion as little as we possible can (remember it's an ugly hack). Instead we want to use real lighting equations and framework. So, in order to do proper sky-dome kind of lighting, put a blue directional light coming from up in the sky pointing downwards (which produces a blue version of the otherwise gray-dead hemispherical occlusion trick), and give it some color variation based on the incident angle. That looks a million times better than the horrible hemispherical occlusion, and it is equally cheap.



Final render (key light added)

Anyway, this is not a lighting tutorial. So, back to the multiresolution occlusion, when you apply it to the ambient lighting, you quickly get results that look acceptable for the little amount of work and embarrassingly simple algorithms/ideas that we have been using. Here you can see the scene rendered with only ambient light using the multiresolution occlusion described in this article, and the same image with the sun light added.

Another view of the same scene:



inigo quilez - learning computer graphics since 1994